

Trainee must identify whether the problem is:

- **SYN** = Syntax error
- **LOG** = Logic error
- **SEM** = Semantic/meaning error

Then the **solution section is completely separate.**

The set progresses from **Basic** → **Medium** → **Hard** and deliberately mixes:

- **SELECT**
- **WHERE**
- comparison operators
- logical operators
- **IN**
- **BETWEEN**
- **LIKE**
- **IS NULL**
- **CASE**
- **DISTINCT**
- **ORDER BY**
- **GROUP BY**
- **HAVING**
- aggregate functions
- all major joins
- self joins
- cross joins
- subqueries
- correlated subqueries
- CTEs
- **UNION**
- **EXISTS**
- **NOT EXISTS**
- window functions
- **ROW_NUMBER**
- **RANK**
- **DENSE_RANK**
- **LAG**
- **LEAD**
- running totals
- moving averages

- partitioning
- conditional aggregation
- top-N problems
- duplicate detection
- NULL handling
- date functions
- string functions
- nested queries
- joins + windows
- joins + aggregates
- subqueries + windows

Compatibility: MySQL 8.0+. Window functions and CTEs require MySQL 8.0 or later.

0. Practice Database

Use the following schema for all 100 questions.

Tables

```
CREATE DATABASE sql_training;  
USE sql_training;
```

```
CREATE TABLE departments (  
    department_id INT PRIMARY KEY,  
    department_name VARCHAR(50),  
    location VARCHAR(50)  
);
```

```
CREATE TABLE employees (  
    employee_id INT PRIMARY KEY,  
    employee_name VARCHAR(100),  
    department_id INT,  
    manager_id INT NULL,  
    salary DECIMAL(10,2),  
    hire_date DATE,  
    job_title VARCHAR(50),  
    FOREIGN KEY (department_id) REFERENCES departments(department_id)  
);
```

```
CREATE TABLE customers (  
    customer_id INT PRIMARY KEY,  
    customer_name VARCHAR(100),  
    city VARCHAR(50),  
    state VARCHAR(50),  
    country VARCHAR(50),  
    segment VARCHAR(30),  
    signup_date DATE,  
    sales_rep_id INT NULL  
);
```

```
CREATE TABLE products (  
    product_id INT PRIMARY KEY,  
    product_name VARCHAR(100),  
    category VARCHAR(50),  
    subcategory VARCHAR(50),  
    price DECIMAL(10,2),  
    cost DECIMAL(10,2),  
    supplier_id INT NULL  
);
```

```
CREATE TABLE orders (  
    order_id INT PRIMARY KEY,  
    customer_id INT,  
    employee_id INT,  
    order_date DATE,  
    status VARCHAR(30),  
    shipping_date DATE NULL,  
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id),  
    FOREIGN KEY (employee_id) REFERENCES employees(employee_id)  
);
```

```
CREATE TABLE order_items (  
    order_item_id INT PRIMARY KEY,  
    order_id INT,  
    product_id INT,  
    quantity INT,  
    unit_price DECIMAL(10,2),  
    discount DECIMAL(5,2),  
    FOREIGN KEY (order_id) REFERENCES orders(order_id),  
    FOREIGN KEY (product_id) REFERENCES products(product_id)  
);
```

PART A — BASIC

Queries 1–35

These concentrate on fundamentals and simple combinations.

Q1 — Basic SELECT

Error type: SEM

```
SELECT customer_id, customer_name, salary  
FROM customers;
```

Task: Find the error.

Q2 — WHERE

Error type: SYN

```
SELECT *  
FROM customers  
WHERE country = 'India'  
AND;
```

Q3 — Comparison Operator

Error type: LOG

```
SELECT *  
FROM products  
WHERE price < 500  
AND price > 1000;
```

Q4 — BETWEEN

Error type: SEM

```
SELECT *  
FROM products  
WHERE price BETWEEN 1000 AND 500;
```

Q5 — IN

Error type: SYN

```
SELECT *  
FROM customers  
WHERE country IN 'India', 'USA';
```

Q6 — LIKE

Error type: LOG

```
SELECT *  
FROM customers  
WHERE customer_name LIKE '%a';
```

Task: Find customers whose names **start with A**.

Q7 — NULL

Error type: SEM

```
SELECT *  
FROM customers  
WHERE sales_rep_id = NULL;
```

Q8 — DISTINCT

Error type: LOG

```
SELECT DISTINCT customer_id, city  
FROM customers;
```

Task: Find distinct cities only.

Q9 — ORDER BY

Error type: SYN

```
SELECT customer_name, city
FROM customers
ORDER BY city DESCENDING;
```

Q10 — LIMIT

Error type: LOG

```
SELECT *
FROM products
ORDER BY price DESC
LIMIT 5, 10;
```

Task: Find the five most expensive products.

Q11 — AND / OR

Error type: LOG

```
SELECT *
FROM products
WHERE category = 'Electronics'
OR category = 'Furniture'
AND price > 1000;
```

Task: Products must be either Electronics or Furniture **and** price > 1000.

Q12 — Parentheses

Error type: LOG

```
SELECT *  
FROM customers  
WHERE country = 'India'  
AND segment = 'Consumer'  
OR segment = 'Corporate';
```

Task: Find Indian customers belonging to Consumer or Corporate.

Q13 — CASE

Error type: SYN

```
SELECT  
    product_name,  
    price,  
    CASE  
        WHEN price >= 1000 THEN 'Expensive'  
        WHEN price >= 500 'Medium'  
        ELSE 'Cheap'  
    END AS price_category  
FROM products;
```

Q14 — Calculated Column

Error type: SEM

```
SELECT  
    product_name,  
    price,  
    cost,  
    price - price AS profit  
FROM products;
```

Q15 — Aggregate

Error type: LOG

```
SELECT COUNT(customer_id)  
FROM customers;
```

Task: Count all customers, including customers where `customer_id` could be NULL.

Q16 — SUM

Error type: SEM

```
SELECT SUM(price)
FROM products
WHERE category = 'Electronics'
GROUP BY product_name;
```

Task: Find total Electronics sales value, not one total per product.

Q17 — AVG

Error type: SYN

```
SELECT AVG(price), category
FROM products
GROUP category;
```

Q18 — GROUP BY

Error type: SEM

```
SELECT category, product_name, AVG(price)
FROM products
GROUP BY category;
```

Q19 — HAVING

Error type: LOG

```
SELECT category, AVG(price) AS avg_price
FROM products
WHERE AVG(price) > 1000
GROUP BY category;
```

Q20 — WHERE vs HAVING

Error type: LOG

```
SELECT category, COUNT(*) AS product_count
FROM products
GROUP BY category
HAVING category = 'Electronics'
AND product_count > 5;
```

JOINS — BASIC

Q21 — INNER JOIN

Error type: SYN

```
SELECT
    c.customer_name,
    o.order_id
FROM customers c
JOIN orders o
ON c.customer_id o.customer_id;
```

Q22 — INNER JOIN + filter

Error type: LOG

```
SELECT
    c.customer_name,
    o.order_id
FROM customers c
JOIN orders o
    ON c.customer_id = o.customer_id
WHERE o.status = 'Completed'
AND c.country = 'India'
OR c.country = 'USA';
```

Task: Completed orders from customers in India or USA.

Q23 — LEFT JOIN

Error type: LOG

```
SELECT
    c.customer_name,
    o.order_id
FROM customers c
LEFT JOIN orders o
    ON c.customer_id = o.customer_id
WHERE o.order_id IS NOT NULL;
```

Task: Return all customers, including customers with no orders.

Q24 — RIGHT JOIN

Error type: SEM

```
SELECT
    c.customer_name,
    o.order_id
FROM customers c
RIGHT JOIN orders o
    ON c.customer_id = o.order_id;
```

Q25 — Three-table JOIN

Error type: SYN

```
SELECT
    c.customer_name,
    o.order_id,
    oi.quantity
FROM customers c
JOIN orders o
    ON c.customer_id = o.customer_id
```

```
JOIN order_items oi
  oi.order_id = o.order_id;
```

Q26 — JOIN Products

Error type: LOG

```
SELECT
  o.order_id,
  p.product_name,
  oi.quantity
FROM orders o
JOIN order_items oi
  ON o.order_id = oi.order_id
JOIN products p
  ON oi.product_id = oi.product_id;
```

Q27 — Four-table JOIN

Error type: SEM

```
SELECT
  c.customer_name,
  o.order_id,
  p.product_name,
  oi.quantity
FROM customers c
JOIN orders o
  ON c.customer_id = o.customer_id
JOIN order_items oi
  ON o.order_id = oi.order_id
JOIN products p
  ON p.product_id = oi.product_id
WHERE oi.quantity = 0;
```

Task: Find orders containing products with quantity greater than zero.

Q28 — LEFT JOIN + COUNT

Error type: LOG

```
SELECT
  c.customer_name,
  COUNT(o.order_id) AS orders
FROM customers c
LEFT JOIN orders o
  ON c.customer_id = o.customer_id
WHERE COUNT(o.order_id) > 2
GROUP BY c.customer_name;
```

Q29 — GROUP BY + JOIN

Error type: SEM

```
SELECT
  c.country,
  SUM(oi.quantity * oi.unit_price) AS revenue
FROM customers c
JOIN orders o
  ON c.customer_id = o.customer_id
JOIN order_items oi
  ON o.order_id = oi.order_id
GROUP BY c.customer_id;
```

Q30 — Employee Department

Error type: LOG

```
SELECT
  e.employee_name,
  d.department_name
FROM employees e
LEFT JOIN departments d
  ON e.department_id = d.department_id
WHERE d.department_name IS NULL;
```

Task: Find employees who belong to a department.

SUBQUERIES — BASIC

Q31

Error type: LOG

Find products more expensive than the average product price.

```
SELECT *  
FROM products  
WHERE price > (  
    SELECT price  
    FROM products  
);
```

Q32

Error type: SYN

```
SELECT *  
FROM products  
WHERE price > SELECT AVG(price) FROM products;
```

Q33

Error type: SEM

Find customers who have placed at least one order.

```
SELECT *  
FROM customers  
WHERE customer_id IN (  
    SELECT order_id  
    FROM orders  
);
```

Q34

Error type: LOG

Find customers who have never placed an order.

```
SELECT *  
FROM customers  
WHERE customer_id IN (  
    SELECT customer_id  
    FROM orders  
);
```

Q35

Error type: SEM

```
SELECT *  
FROM products p  
WHERE EXISTS (  
    SELECT product_id  
    FROM order_items oi  
);
```

Task: Find products that have actually been ordered.

PART B — MEDIUM

Queries 36–70

Now the trainee must combine multiple concepts.

Q36 — Aggregation + JOIN

Error type: LOG

```
SELECT  
    c.customer_name,  
    SUM(oi.quantity * oi.unit_price) AS total_sales  
FROM customers c
```

```
JOIN orders o
  ON c.customer_id = o.customer_id
JOIN order_items oi
  ON o.order_id = oi.order_id
GROUP BY c.customer_id;
```

Q37 — Revenue by product

Error type: SEM

```
SELECT
  p.product_name,
  SUM(oi.quantity * oi.unit_price) AS revenue
FROM products p
JOIN order_items oi
  ON p.product_id = oi.product_id
GROUP BY p.category;
```

Q38 — HAVING

Error type: LOG

```
SELECT
  c.customer_name,
  SUM(oi.quantity * oi.unit_price) AS total_sales
FROM customers c
JOIN orders o
  ON c.customer_id = o.customer_id
JOIN order_items oi
  ON o.order_id = oi.order_id
GROUP BY c.customer_name
HAVING total_sales > 10000;
```

Q39 — Conditional Aggregation

Error type: LOG

```
SELECT
  customer_id,
  SUM(status = 'Completed') AS completed_orders
```

```
FROM orders
GROUP BY customer_id;
```

Task: Count completed orders using conditional aggregation.

Q40 — Date Filter

Error type: SEM

```
SELECT *
FROM orders
WHERE order_date BETWEEN '2026-01-01' AND '2026-01-31'
AND shipping_date <= order_date;
```

Task: Find orders shipped after the order date.

Q41 — DATE functions

Error type: SYN

```
SELECT
    YEAR(order_date) AS year,
    MONTH(order_date) AS month,
    COUNT(*) AS orders
FROM orders
GROUP BY YEAR(order_date), MONTH(order_date)
ORDER BY year month;
```

Q42 — GROUP BY expression

Error type: LOG

```
SELECT
    YEAR(order_date) AS order_year,
    COUNT(*) AS total_orders
FROM orders
GROUP BY order_date;
```

Q43 — Product margin

Error type: LOG

```
SELECT
  product_name,
  price,
  cost,
  price + cost AS margin
FROM products;
```

Q44 — Discount

Error type: SEM

Assume **discount** is percentage.

```
SELECT
  product_id,
  quantity * unit_price AS gross_sales,
  quantity * unit_price - discount AS net_sales
FROM order_items;
```

Q45 — DISTINCT customers

Error type: LOG

```
SELECT COUNT(customer_id)
FROM orders;
```

Task: Count distinct customers who ordered.

JOIN COMBINATIONS

Q46 — LEFT JOIN + aggregation

Error type: LOG

```
SELECT
    c.customer_name,
    COUNT(*) AS order_count
FROM customers c
LEFT JOIN orders o
    ON c.customer_id = o.customer_id
GROUP BY c.customer_name;
```

Q47 — Customers with zero orders

Error type: LOG

```
SELECT
    c.customer_name
FROM customers c
LEFT JOIN orders o
    ON c.customer_id = o.customer_id
WHERE o.order_id = NULL;
```

Q48 — Customers with multiple orders

Error type: SYN

```
SELECT
    c.customer_name,
    COUNT(o.order_id)
FROM customers c
JOIN orders o
    ON c.customer_id = o.customer_id
GROUP BY c.customer_name
HAVING COUNT(o.order_id) >;
```

Q49 — Employees + managers

Self join.

Error type: SEM

```
SELECT
    e.employee_name,
```

```
m.employee_name AS manager_name
FROM employees e
JOIN employees m
  ON e.employee_id = m.manager_id;
```

Q50 — Employee's manager

Error type: LOG

```
SELECT
  e.employee_name,
  m.employee_name AS manager_name
FROM employees e
LEFT JOIN employees m
  ON e.manager_id = e.employee_id;
```

Q51 — Department salary

Error type: LOG

```
SELECT
  d.department_name,
  AVG(e.salary) AS avg_salary
FROM departments d
LEFT JOIN employees e
  ON d.department_id = e.department_id
WHERE e.salary > 50000
GROUP BY d.department_name;
```

Q52 — Departments without employees

Error type: LOG

```
SELECT d.department_name
FROM departments d
INNER JOIN employees e
  ON d.department_id = e.department_id
WHERE e.employee_id IS NULL;
```

Q53 — CROSS JOIN

Error type: SEM

```
SELECT
  c.customer_name,
  p.product_name
FROM customers c
CROSS JOIN products p
ON c.customer_id = p.product_id;
```

Q54 — FULL OUTER JOIN simulation

MySQL does not directly support **FULL OUTER JOIN**.

Error type: SYN

```
SELECT *
FROM customers c
FULL OUTER JOIN orders o
ON c.customer_id = o.customer_id;
```

SET OPERATORS

Q55 — UNION

Error type: SEM

```
SELECT customer_id
FROM customers
```

UNION

```
SELECT customer_name
FROM customers;
```

Q56 — UNION ALL

Error type: LOG

```
SELECT customer_id  
FROM customers
```

```
UNION ALL
```

```
SELECT customer_id  
FROM orders;
```

Task: Return customer IDs appearing in either table.

Q57 — INTERSECT alternative

MySQL 8.0 does not support **INTERSECT** in the traditional MySQL approach.

Error type: SYN

```
SELECT customer_id  
FROM customers  
INTERSECT  
SELECT customer_id  
FROM orders;
```

SUBQUERIES

Q58 — Correlated subquery

Error type: LOG

Find products whose price is greater than the average price of their own category.

```
SELECT *  
FROM products p  
WHERE price > (  
    SELECT AVG(price)  
    FROM products  
);
```

Q59

Error type: SEM

Find employees earning above their department average.

```
SELECT *  
FROM employees e  
WHERE salary > (  
    SELECT AVG(salary)  
    FROM employees x  
    WHERE x.department_id = e.department_id  
);
```

Q60 — EXISTS

Error type: LOG

```
SELECT *  
FROM customers c  
WHERE EXISTS (  
    SELECT 1  
    FROM orders o  
    WHERE o.customer_id != c.customer_id  
);
```

Q61 — NOT EXISTS

Error type: LOG

Find products never ordered.

```
SELECT *  
FROM products p  
WHERE NOT EXISTS (  
    SELECT 1  
    FROM order_items oi  
    WHERE oi.product_id = p.product_id  
);
```

Q62 — MAX subquery

Error type: LOG

Find the highest-paid employee.

```
SELECT *  
FROM employees  
WHERE salary < (  
    SELECT MAX(salary)  
    FROM employees  
);
```

Q63 — Second highest salary

Error type: LOG

```
SELECT MAX(salary)  
FROM employees  
WHERE salary < MAX(salary);
```

WINDOW FUNCTIONS

Q64 — ROW_NUMBER

Error type: SYN

```
SELECT  
    employee_name,  
    salary,  
    ROW_NUMBER OVER (  
        ORDER BY salary DESC  
    ) AS rn  
FROM employees;
```

Q65 — RANK

Error type: LOG

```
SELECT
    employee_name,
    salary,
    RANK() OVER (
        ORDER BY salary ASC
    ) AS salary_rank
FROM employees;
```

Task: Highest salary should have rank 1.

Q66 — PARTITION BY

Error type: SEM

```
SELECT
    employee_name,
    department_id,
    salary,
    RANK() OVER (
        PARTITION BY employee_id
        ORDER BY salary DESC
    ) AS dept_rank
FROM employees;
```

Q67 — Department ranking

Error type: LOG

```
SELECT
    employee_name,
    department_id,
    salary,
    RANK() OVER (
        ORDER BY salary DESC
    ) AS dept_rank
FROM employees;
```

Task: Rank employees within each department.

Q68 — DENSE_RANK

Error type: LOG

```
SELECT
    employee_name,
    salary,
    DENSE_RANK() OVER (
        ORDER BY salary DESC
    ) AS salary_rank
FROM employees
WHERE salary_rank <= 3;
```

Q69 — LAG

Error type: SYN

```
SELECT
    order_id,
    order_date,
    LAG(order_date) OVER
        ORDER BY order_date
    AS previous_date
FROM orders;
```

Q70 — Running total

Error type: LOG

```
SELECT
    order_date,
    SUM(total_sales) OVER (
        ORDER BY order_date
    ) AS running_sales
FROM orders;
```

PART C — HARD

Queries 71–100

These combine joins, CTEs, subqueries, windows, aggregates and business logic.

Q71 — Top 3 employees per department

Error type: LOG

```
SELECT *
FROM (
    SELECT
        employee_name,
        department_id,
        salary,
        RANK() OVER (
            ORDER BY salary DESC
        ) AS rnk
    FROM employees
) x
WHERE rnk <= 3;
```

Q72 — Correct partition but wrong ranking

Error type: LOG

```
SELECT *
FROM (
    SELECT
        employee_name,
        department_id,
        salary,
        RANK() OVER (
            PARTITION BY department_id
            ORDER BY salary ASC
        ) AS rnk
    FROM employees
) x
WHERE rnk <= 3;
```

Task: Top three highest-paid employees per department.

Q73 — Top product per category

Error type: LOG

```
SELECT *
FROM (
  SELECT
    product_name,
    category,
    price,
    ROW_NUMBER() OVER (
      PARTITION BY category
      ORDER BY price ASC
    ) AS rn
  FROM products
) x
WHERE rn = 1;
```

Q74 — Revenue ranking

Error type: SEM

```
SELECT
  p.product_name,
  SUM(oi.quantity * oi.unit_price) AS revenue,
  RANK() OVER (
    ORDER BY revenue DESC
  ) AS revenue_rank
FROM products p
JOIN order_items oi
  ON p.product_id = oi.product_id
GROUP BY p.product_name;
```

Q75 — Customer ranking

Error type: LOG

```
SELECT
  c.customer_name,
```

```
SUM(oi.quantity * oi.unit_price) AS total_sales,  
RANK() OVER (  
    PARTITION BY c.customer_id  
    ORDER BY SUM(oi.quantity * oi.unit_price) DESC  
) AS rnk  
FROM customers c  
JOIN orders o  
    ON c.customer_id = o.customer_id  
JOIN order_items oi  
    ON o.order_id = oi.order_id  
GROUP BY c.customer_id, c.customer_name;
```

Task: Rank customers globally by sales.

Q76 — LAG sales

Error type: LOG

```
SELECT  
    order_date,  
    SUM(oi.quantity * oi.unit_price) AS daily_sales,  
    LAG(SUM(oi.quantity * oi.unit_price))  
        OVER (ORDER BY order_date) AS previous_sales  
FROM orders o  
JOIN order_items oi  
    ON o.order_id = oi.order_id  
GROUP BY order_date;
```

Task: Find the difference from previous day's sales.

Q77 — LEAD

Error type: SYN

```
SELECT  
    order_date,  
    LEAD(order_date) OVER (  
        ORDER BY order_date  
    ) AS next_order_date  
FROM orders
```

Q78 — Running revenue

Error type: LOG

```
SELECT
    o.order_date,
    SUM(oi.quantity * oi.unit_price) AS daily_sales,
    SUM(oi.quantity * oi.unit_price)
      OVER (ORDER BY o.order_date) AS running_sales
FROM orders o
JOIN order_items oi
  ON o.order_id = oi.order_id
GROUP BY o.order_date;
```

Q79 — Moving average

Error type: LOG

```
SELECT
    order_date,
    daily_sales,
    AVG(daily_sales) OVER (
        ORDER BY order_date
        ROWS BETWEEN 7 PRECEDING AND CURRENT ROW
    ) AS moving_avg
FROM daily_sales;
```

Assume `daily_sales` is not a table.

Q80 — CTE

Error type: SYN

```
WITH customer_sales AS
SELECT
    c.customer_id,
    SUM(oi.quantity * oi.unit_price) AS total_sales
FROM customers c
```

```
JOIN orders o
ON c.customer_id = o.customer_id
JOIN order_items oi
ON o.order_id = oi.order_id
GROUP BY c.customer_id
```

```
SELECT *
FROM customer_sales;
```

Q81 — CTE + HAVING

Error type: LOG

```
WITH customer_sales AS (
  SELECT
    c.customer_id,
    SUM(oi.quantity * oi.unit_price) AS total_sales
  FROM customers c
  JOIN orders o
  ON c.customer_id = o.customer_id
  JOIN order_items oi
  ON o.order_id = oi.order_id
  GROUP BY c.customer_id
)
SELECT *
FROM customer_sales
WHERE total_sales > 10000;
```

Q82 — Customer above average

Error type: LOG

```
WITH customer_sales AS (
  SELECT
    customer_id,
    SUM(oi.quantity * oi.unit_price) AS total_sales
  FROM orders o
  JOIN order_items oi
  ON o.order_id = oi.order_id
  GROUP BY customer_id
)
```

```
SELECT *
FROM customer_sales
WHERE total_sales > AVG(total_sales);
```

Q83 — CTE + ranking

Error type: LOG

```
WITH customer_sales AS (
  SELECT
    c.customer_id,
    c.customer_name,
    SUM(oi.quantity * oi.unit_price) AS total_sales
  FROM customers c
  JOIN orders o
    ON c.customer_id = o.customer_id
  JOIN order_items oi
    ON o.order_id = oi.order_id
  GROUP BY c.customer_id, c.customer_name
)
SELECT
  *,
  RANK() OVER (
    PARTITION BY customer_id
    ORDER BY total_sales DESC
  ) AS rnk
FROM customer_sales;
```

Q84 — Department average

Error type: LOG

```
SELECT
  e.employee_name,
  e.department_id,
  e.salary,
  AVG(e.salary) OVER () AS dept_avg
FROM employees e;
```

Q85 — Salary difference from department average

Error type: LOG

```
SELECT
    employee_name,
    department_id,
    salary,
    salary - AVG(salary) OVER () AS difference
FROM employees;
```

Q86 — Employee salary percentage

Error type: LOG

```
SELECT
    employee_name,
    salary,
    salary / SUM(salary) OVER () AS salary_percentage
FROM employees;
```

Task: Percentage of total departmental salary.

Q87 — Duplicate customers

Error type: SEM

```
SELECT customer_name, COUNT(*)
FROM customers
GROUP BY customer_id
HAVING COUNT(*) > 1;
```

Q88 — Duplicate detection using window

Error type: LOG

```
SELECT *
FROM (
    SELECT
        customer_name,
        ROW_NUMBER() OVER (
```

```
        PARTITION BY customer_id
        ORDER BY customer_name
    ) AS rn
FROM customers
) x
WHERE rn = 2;
```

Q89 — Latest order per customer

Error type: LOG

```
SELECT
    customer_id,
    order_id,
    order_date
FROM (
    SELECT
        customer_id,
        order_id,
        order_date,
        ROW_NUMBER() OVER (
            PARTITION BY customer_id
            ORDER BY order_date ASC
        ) AS rn
    FROM orders
) x
WHERE rn = 1;
```

Q90 — Second highest salary per department

Error type: LOG

```
SELECT *
FROM (
    SELECT
        employee_name,
        department_id,
        salary,
        DENSE_RANK() OVER (
            PARTITION BY department_id
            ORDER BY salary DESC
        ) AS rn
    FROM employees
) x
WHERE rn = 2;
```

```
    ) AS rnk  
FROM employees  
) x  
WHERE rnk = 1;
```

ADVANCED COMBINATIONS

Q91 — Customer's first and last order

Error type: LOG

```
SELECT  
    customer_id,  
    FIRST_VALUE(order_date) OVER (  
        PARTITION BY customer_id  
        ORDER BY order_date DESC  
    ) AS first_order,  
    LAST_VALUE(order_date) OVER (  
        PARTITION BY customer_id  
        ORDER BY order_date  
    ) AS last_order  
FROM orders;
```

Q92 — Orders between first and last order

Error type: SEM

```
SELECT  
    customer_id,  
    COUNT(*) AS orders  
FROM orders  
GROUP BY customer_id  
HAVING order_date BETWEEN  
    MIN(order_date) AND MAX(order_date);
```

Q93 — Customer order sequence

Error type: LOG

```
SELECT
    customer_id,
    order_id,
    order_date,
    ROW_NUMBER() OVER (
        ORDER BY order_date
    ) AS order_number
FROM orders;
```

Task: Number orders separately for each customer.

Q94 — Time between orders

Error type: SEM

```
SELECT
    customer_id,
    order_date,
    DATEDIFF(
        order_date,
        LAG(order_date) OVER (
            PARTITION BY customer_id
            ORDER BY order_date
        )
    ) AS days_between
FROM orders;
```

Q95 — Customers with increasing order values

Error type: LOG

```
SELECT
    customer_id,
    order_id,
    order_value,
    LAG(order_value) OVER (
        ORDER BY order_date
    ) AS previous_value
FROM orders;
```

Assume `order_value` is not an existing column.

Q96 — Highest-selling category

Error type: LOG

```
SELECT
  p.category,
  SUM(oi.quantity * oi.unit_price) AS revenue
FROM products p
JOIN order_items oi
  ON p.product_id = oi.product_id
GROUP BY p.category
ORDER BY revenue ASC
LIMIT 1;
```

Q97 — Category percentage of total revenue

Error type: LOG

```
SELECT
  p.category,
  SUM(oi.quantity * oi.unit_price) AS revenue,
  SUM(oi.quantity * oi.unit_price)
    / SUM(SUM(oi.quantity * oi.unit_price))
    OVER (PARTITION BY p.category) AS percentage
FROM products p
JOIN order_items oi
  ON p.product_id = oi.product_id
GROUP BY p.category;
```

Q98 — Top customer in each country

Error type: LOG

```
WITH customer_sales AS (
  SELECT
    c.customer_id,
    c.customer_name,
    c.country,
    SUM(oi.quantity * oi.unit_price) AS sales
```

```

FROM customers c
JOIN orders o
  ON c.customer_id = o.customer_id
JOIN order_items oi
  ON o.order_id = oi.order_id
GROUP BY
  c.customer_id,
  c.customer_name,
  c.country
)
SELECT *
FROM (
  SELECT
    *,
    RANK() OVER (
      ORDER BY sales DESC
    ) AS rnk
  FROM customer_sales
) x
WHERE rnk = 1;

```

Q99 — Employees earning above department average AND company average

Error type: LOG

```

SELECT
  employee_name,
  department_id,
  salary
FROM employees e
WHERE salary > (
  SELECT AVG(salary)
  FROM employees
  WHERE department_id = e.department_id
)
AND salary > (
  SELECT AVG(salary)
  FROM employees
  WHERE department_id = e.department_id
);

```

Q100 — Advanced sales analysis

Error type: LOG

Find the **top 3 customers in each country based on total revenue**, including:

- customer
- country
- revenue
- rank
- percentage of country revenue

```
WITH customer_sales AS (  
  SELECT  
    c.customer_id,  
    c.customer_name,  
    c.country,  
    SUM(oi.quantity * oi.unit_price) AS revenue  
  FROM customers c  
  JOIN orders o  
    ON c.customer_id = o.customer_id  
  JOIN order_items oi  
    ON o.order_id = oi.order_id  
  GROUP BY  
    c.customer_id,  
    c.customer_name,  
    c.country  
)  
SELECT  
  customer_name,  
  country,  
  revenue,  
  RANK() OVER (  
    ORDER BY revenue DESC  
  ) AS country_rank,  
  revenue /  
    SUM(revenue) OVER () * 100 AS country_percentage  
FROM customer_sales  
WHERE country_rank <= 3;
```